SQL JOINS ALL IN ONE :

12 September 2025 15:35

🎓 SQL JOINS (Full Explanation for Students)

🔷 1. Introduction:

In databases, data is often stored in **different tables** for better organization.
👉 A **JOIN** allows us to combine data from two or more tables using a related column (usually a primary key and foreign key).

🔷 2. Real-life Example:

Imagine a **school database**:
- One table stores **students**.
- Another table stores **classes**.

If you want to know:
👉 "Which student is in which class?"
You need to **JOIN** these two tables.

🔷 3. Sample Tables:

Students

student_id	name	class_id
1	Alice	101
2	Bob	102
3	Charlie	103
4	David	NULL

Classes

class_id	class_name
101	Math
102	Science
104	English

🔷 4. Types of JOINS

1️⃣ INNER JOIN → Only matching records:

👉 Students who are assigned to a class that exists.

```
SELECT students.name, classes.class_name
FROM students
INNER JOIN classes
ON students.class_id = classes.class_id;
```

✅ Result:

name	class_name

Alice	Math
Bob	Science

Charlie (103 doesn't exist in classes) and David (NULL) are excluded.

2 LEFT JOIN → All students, with classes if available:

👉 We want **all students** even if they don't have a class yet.

```
SELECT students.name, classes.class_name
FROM students
LEFT JOIN classes
ON students.class_id = classes.class_id;
```

✅ Result:

name	class_name
Alice	Math
Bob	Science
Charlie	NULL
David	NULL

3 RIGHT JOIN → All classes, with students if available:

👉 We want **all classes** even if no student is enrolled.

```
SELECT students.name, classes.class_name
FROM students
RIGHT JOIN classes
ON students.class_id = classes.class_id;
```

✅ Result:

name	class_name
Alice	Math
Bob	Science
NULL	English

4 FULL OUTER JOIN → All students + All classes:

👉 MySQL doesn't support it directly, but we can use UNION.

```
SELECT students.name, classes.class_name
FROM students
LEFT JOIN classes ON students.class_id = classes.class_id

UNION
SELECT students.name, classes.class_name
FROM students
RIGHT JOIN classes ON students.class_id = classes.class_id;
```

✅ Result:

name	class_name

Alice	Math
Bob	Science
Charlie	NULL
David	NULL
NULL	English

5 CROSS JOIN → Every possible combination:

👉 Every student paired with every class.

```
SELECT students.name, classes.class_name
FROM students
CROSS JOIN classes;
```

✅ Result (4 students × 3 classes = 12 rows):

name	class_name
Alice	Math
Alice	Science
Alice	English
Bob	Math
...	...

6 SELF JOIN → Join table with itself:

👉 Example: Employees and their managers (same table).

Employees

emp_id	name	manager_id
1	Alice	NULL
2	Bob	1
3	Charlie	1
4	David	2

```
SELECT e1.name AS employee, e2.name AS manager
FROM employees e1
LEFT JOIN employees e2
ON e1.manager_id = e2.emp_id;
```

✅ Result:

employee	manager
Alice	NULL
Bob	Alice
Charlie	Alice
David	Bob

🔷 5. Easy Analogies:

- **INNER JOIN** → People who play both **Cricket & Football**.
- **LEFT JOIN** → All Cricket players (even if they don't play Football).

- **RIGHT JOIN** → All Football players (even if they don't play Cricket).
- **FULL OUTER JOIN** → Anyone in **Cricket OR Football**.
- **CROSS JOIN** → Every Cricket player paired with every Football player.
- **SELF JOIN** → Family tree → Child & Parent in same table.